

```

...
#include "SampleConfiguration.h"
...
bool SampleState::isClipboardContentChangedEnabled = false;
...
bool SampleState::IsClipboardContentChangedEnabled()
{
    return isClipboardContentChangedEnabled;
}
void SampleState::IsClipboardContentChangedEnabled(bool checked)
{
    if (isClipboardContentChangedEnabled != checked)
    {
        isClipboardContentChangedEnabled = checked;
    }
}
}

```

Again, a field with `static` storage (such as `SampleState::isClipboardContentChangedEnabled`) must be defined once in the application, and a `.cpp` file is a good place for that (`SampleConfiguration.cpp` in this case).

BuildClipboardFormatsOutputString

This method is a public member of `MainPage`, and it's defined in `SampleConfiguration.cs`.

```

C#

// SampleConfiguration.cs
...
public string BuildClipboardFormatsOutputString()
{
    DataPackageView clipboardContent =
Windows.ApplicationModel.DataTransfer.Clipboard.GetContent();
    StringBuilder output = new StringBuilder();

    if (clipboardContent != null && clipboardContent.AvailableFormats.Count
> 0)
    {
        output.Append("Available formats in the clipboard:");
        foreach (var format in clipboardContent.AvailableFormats)
        {
            output.Append(Environment.NewLine + " * " + format);
        }
    }
    else
    {
        output.Append("The clipboard is empty");
    }
}

```

```
    return output.ToString();  
}  
...
```

In C++/WinRT, we'll make **BuildClipboardFormatsOutputString** a public static method of **SampleState**. We can make it `static` because it doesn't access any instance members.

To use the **Clipboard** and **DataPackageView** types in C++/WinRT, we'll need to include the C++/WinRT Windows namespace header file

```
winrt/Windows.ApplicationModel.DataTransfer.h.
```

In C#, the **DataPackageView.AvailableFormats** property is an **ReadOnlyList**, so we can access the **Count** property of that. In C++/WinRT, the **DataPackageView::AvailableFormats** accessor function returns an **IVectorView**, which has a **Size** accessor function that we can call.

To port the use of the C# **System.Text.StringBuilder** type, we'll make use of the standard C++ type `std::wostream`. That type is an output stream for wide strings (and to use it we'll need to include the `sstream` header file). Instead of using an **Append** method like you do with a **StringBuilder**, you use the **insertion operator** (`<<`) with an output stream such as `wostream`. For more info, see [iostream programming](#), and [Formatting C++/WinRT strings](#).

The C# code constructs a **StringBuilder** with the `new` keyword. In C#, objects are reference types by default, declared on the heap with `new`. In modern standard C++, objects are value types by default, declared on the stack (without using `new`). So we port `StringBuilder output = new StringBuilder();` to C++/WinRT as simply `std::wostream output;`

The C# `var` keyword asks the compiler to infer a type. You port `var` to `auto` in C++/WinRT. But in C++/WinRT, there are cases where (in order to avoid copies) you want a *reference* to an inferred (or deduced) type, and you express an lvalue reference to a deduced type with `auto&`. There are also cases where you want a special kind of reference that binds correctly whether it's initialized with an *lvalue* or with an *rvalue*. And you express that with `auto&&`. That's the form that you see used in the `for` loop in the ported code below. For an introduction to *lvalues* and *rvalues*, see [Value categories, and references to them](#).

Edit `pch.h`, `SampleConfiguration.h`, and `SampleConfiguration.cpp` to match the listings below.

```

// pch.h
...
#include <sstream>
#include "winrt/Windows.ApplicationModel.DataTransfer.h"
...

// SampleConfiguration.h
...
struct SampleState
{
    static hstring BuildClipboardFormatsOutputString();
    ...
}
...

// SampleConfiguration.cpp
...
using namespace Windows::ApplicationModel::DataTransfer;
...
hstring SampleState::BuildClipboardFormatsOutputString()
{
    DataPackageView clipboardContent{ Clipboard::GetContent() };
    std::wostringstream output;

    if (clipboardContent && clipboardContent.AvailableFormats().Size() > 0)
    {
        output << L"Available formats in the clipboard:";
        for (auto&& format : clipboardContent.AvailableFormats())
        {
            output << std::endl << L" * " << std::wstring_view(format);
        }
    }
    else
    {
        output << L"The clipboard is empty";
    }

    return hstring{ output.str() };
}

```

ⓘ Note

The syntax in the line of code `DataPackageView clipboardContent{ Clipboard::GetContent() }` uses a feature of modern standard C++ called *uniform initialization*, with its characteristic use of curly brackets instead of an `=` sign. That syntax makes it clear that initialization, rather than assignment, is taking place. If you prefer the form of syntax that *looks* like assignment (but actually isn't), then you can replace the syntax above with the equivalent `DataPackageView`

```
clipboardContent = Clipboard::GetContent();
```

 It's a good idea to become comfortable with both ways of expressing initialization, though, because you're likely to see both used frequently in the code you encounter.

DisplayToast

DisplayToast is a public static method of the C# **MainPage** class, and you'll find it defined in `SampleConfiguration.cs`. In C++/WinRT, we'll make it a public static method of **SampleState**.

We've already encountered most of the details and techniques that are relevant to porting this method. One new item to note is that you port a C# verbatim string literal (`@`) to a standard C++ [raw string literal](#) (`LR`).

Also, when you reference the [ToastNotification](#) and [XmlDocument](#) types in C++/WinRT, you can either qualify them by namespace name, or you can edit `SampleConfiguration.cpp` and add `using namespace` directives such as the following example.

C++/WinRT

```
using namespace Windows::UI::Notifications;
```

You have the same choice when you reference the [XmlDocument](#) type, and whenever you reference any other Windows Runtime type.

Apart from those items, just follow the same guidance that you did previously to accomplish the following steps.

- Declare the method in `SampleConfiguration.h`, and define it in `SampleConfiguration.cpp`.
- Edit `pch.h` to include any necessary C++/WinRT Windows namespace header files.
- Construct C++/WinRT objects on the stack, not on the heap.
- Replace calls to property get accessors with function-call syntax `( )`.

A very common cause of compiler/linker errors is forgetting to include the C++/WinRT Windows namespace header files that you need. For more info about one possible error, see [C3779: Why is the compiler giving me a "consume_Something: function that returns 'auto' cannot be used before it is defined" error?](#).

If you want to follow along with the walkthrough and port **DisplayToast** yourself, then you can compare your results to the code in the C++/WinRT version in the ZIP of the

[Clipboard sample](#) source code that you downloaded.

EnableClipboardContentChangedNotifications

`EnableClipboardContentChangedNotifications` is a public static method of the C# `MainPage` class, and it's defined in `SampleConfiguration.cs`.

C#

```
// SampleConfiguration.cs
...
public bool EnableClipboardContentChangedNotifications(bool enable)
{
    if (IsClipboardContentChangedEnabled == enable)
    {
        return false;
    }

    IsClipboardContentChangedEnabled = enable;
    if (enable)
    {
        Clipboard.ContentChanged += OnClipboardChanged;
        Window.Current.Activated += OnWindowActivated;
    }
    else
    {
        Clipboard.ContentChanged -= OnClipboardChanged;
        Window.Current.Activated -= OnWindowActivated;
    }
    return true;
}
...
private void OnClipboardChanged(object sender, object e) { ... }
private void OnWindowActivated(object sender, WindowActivatedEventArgs e) {
    ... }
...
```

In C++/WinRT, we'll make it a public static method of `SampleState`.

In C#, you use the `+=` and `-=` operator syntax to register and revoke event-handling delegates. In C++/WinRT, you have several syntactic options to register/revoke a delegate, as described in [Handle events by using delegates in C++/WinRT](#). But the general form is that you register and revoke with calls to a pair of functions named for the event. To register, you pass your delegate to the registering function, and you retrieve a revocation token in return (a `winrt::event_token`). To revoke, you pass that token to the revocation function. In this case, the handler is static and (as you can see in the following code listing) the function call syntax is straightforward.

Similar tokens *are* actually used, behind the scenes, in C#. But the language makes that detail implicit. C++/WinRT makes it explicit.

The **object** type appears in the C# event handler signatures. In the C# language, **object** is an [alias](#) for the .NET [System.Object](#) type. The equivalent in C++/WinRT is [winrt::Windows::Foundation::IInspectable](#). So, you'll see **IInspectable** in the C++/WinRT event handlers.

Edit `SampleConfiguration.h` and `SampleConfiguration.cpp` to match the listings below.

C++/WinRT

```
// SampleConfiguration.h
...
    static bool EnableClipboardContentChangedNotifications(bool enable);
    ...
private:
    ...
    static event_token clipboardContentChangedToken;
    static event_token activatedToken;
    static void OnClipboardChanged(Windows::Foundation::IInspectable const& sender, Windows::Foundation::IInspectable const& e);
    static void OnWindowActivated(Windows::Foundation::IInspectable const& sender, Windows::UI::Core::WindowActivatedEventArgs const& e);
    ...

// SampleConfiguration.cpp
...
using namespace Windows::Foundation;
using namespace Windows::UI::Core;
using namespace Windows::UI::Xaml;
...
event_token SampleState::clipboardContentChangedToken;
event_token SampleState::activatedToken;
...
bool SampleState::EnableClipboardContentChangedNotifications(bool enable)
{
    if (isClipboardContentChangedEnabled == enable)
    {
        return false;
    }

    IsClipboardContentChangedEnabled(enable);
    if (enable)
    {
        clipboardContentChangedToken =
Clipboard::ContentChanged(OnClipboardChanged);
        activatedToken = Window::Current().Activated(OnWindowActivated);
    }
    else
    {
        Clipboard::ContentChanged(clipboardContentChangedToken);
    }
}
```

```

        Window::Current().Activated(activatedToken);
    }
    return true;
}
void SampleState::OnClipboardChanged(IInspectable const&, IInspectable
const&){}
void SampleState::OnWindowActivated(IInspectable const&,
WindowActivatedEventArgs const& e){}

```

Leave the event-handling delegates themselves (**OnClipboardChanged** and **OnWindowActivated**) as stubs for now. They're already on our list of members to port, so we'll get to them in later subsections.

OnNavigatedTo

OnNavigatedTo is a protected method of the C# **MainPage** class, and it's defined in `MainPage.xaml.cs`. Here it is, together with XAML **ListBox** that it references.

XAML

```

<!-- MainPage.xaml -->
...
<ListBox x:Name="ScenarioControl" ... />
...

```

C#

```

// MainPage.xaml.cs
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    // Populate the scenario list from the SampleConfiguration.cs file
    var itemCollection = new List<Scenario>();
    int i = 1;
    foreach (Scenario s in scenarios)
    {
        itemCollection.Add(new Scenario { Title = $"{i++} {s.Title}",
ClassType = s.ClassType });
    }
    ScenarioControl.ItemsSource = itemCollection;

    if (Window.Current.Bounds.Width < 640)
    {
        ScenarioControl.SelectedIndex = -1;
    }
    else
    {
        ScenarioControl.SelectedIndex = 0;
    }
}

```

```

    }
}

```

It's an important and interesting method, because here's where our collection of **Scenario** objects is assigned to the UI. The C# code builds a [System.Collections.Generic.List](#) of **Scenario** objects, and assigns that to the [ItemsSource](#) property of a **ListBox** (which is an items control). And, in C#, we use [string interpolation](#) to build the title for each **Scenario** object (note the use of the `$` special character).

In C++/WinRT, we'll make **OnNavigatedTo** a public method of **MainPage**. And we'll add a stub **ListBox** element to the XAML so that a build will succeed. After the code listing, we'll examine some of the details.

XAML

```

<!-- MainPage.xaml -->
...
<StackPanel ...>
    ...
    <ListBox x:Name="ScenarioControl" />
</StackPanel>
...

```

C++/WinRT

```

// MainPage.h
...
void OnNavigatedTo(Windows::UI::Xaml::Navigation::NavigationEventArgs const&
e);
...

// MainPage.cpp
...
using namespace winrt::Windows::UI::Xaml;
using namespace winrt::Windows::UI::Xaml::Navigation;
...
void MainPage::OnNavigatedTo(NavigationEventArgs const& /* e */)
{
    auto itemCollection =
winrt::single_threaded_observable_vector<IInspectable>();
    int i = 1;
    for (auto s : MainPage::scenarios())
    {
        s.Title = winrt::to_hstring(i++) + L") " + s.Title;
        itemCollection.Append(winrt::box_value(s));
    }
    ScenarioControl().ItemsSource(itemCollection);
}

```



```

if (Window::Current().Bounds().Width < 640)
{
    ScenarioControl().SelectedIndex(-1);
}
else
{
    ScenarioControl().SelectedIndex(0);
}
}
...

```

Again, we're calling the `wintr::single_threaded_observable_vector` function, but this time to create a collection of `IInspectable`. That was part of the decision we made to perform the boxing of our `Scenario` objects on a just-in-time basis.

And, in place of C#'s use of `string interpolation` here, we use a combination of the `to_hstring` function and the `concatenation operator` of `wintr::hstring`.

isApplicationWindowActive

In C#, `isApplicationWindowActive` is a simple private `bool` field belonging to the `MainPage` class, and it's defined in `SampleConfiguration.cs`. It defaults to `false`. In C++/WinRT, we'll make it a private static field of `SampleState` (for the reasons we've already described) in the `SampleConfiguration.h` and `SampleConfiguration.cpp` files, with the same default.

We've already seen how to declare, define, and initialize a static field. For a refresher, look back to what we did with the `isClipboardContentChangedEnabled` field, and do the same with `isApplicationWindowActive`.

needToPrintClipboardFormat

Same pattern as `isApplicationWindowActive` (see the heading immediately before this one).

Button_Click

`Button_Click` is a private (event-handling) method of the C# `MainPage` class, and it's defined in `MainPage.xaml.cs`. Here it is, together with the XAML `SplitView` that it references, and the `ToggleButton` that registers it.

XAML

```

<!-- MainPage.xaml -->
...
<SplitView x:Name="Splitter" ... />
...
<ToggleButton Click="Button_Click" .../>
...

```

C#

```

private void Button_Click(object sender, RoutedEventArgs e)
{
    Splitter.IsPaneOpen = !Splitter.IsPaneOpen;
}

```

And here's the equivalent, ported to C++/WinRT. Note that in the C++/WinRT version, the event handler is `public` (as you can see, you declare it *before* the `private:` declarations). This is because an event handler that's registered in XAML markup, like this one is, needs to be `public` in C++/WinRT in order for the XAML markup to access it. On the other hand, if you register an event handler in imperative code (like we did in `MainPage::EnableClipboardContentChangedNotifications` earlier), then the event handler doesn't need to be `public`.

XAML

```

<!-- MainPage.xaml -->
...
<StackPanel ...>
    ...
    <SplitView x:Name="Splitter" />
</StackPanel>
...

```

C++/WinRT

```

// MainPage.h
...
void Button_Click(Windows::Foundation::IIInspectable const& sender,
Windows::UI::Xaml::RoutedEventArgs const& e);
private:
...

// MainPage.cpp
void MainPage::Button_Click(Windows::Foundation::IIInspectable const& /*
sender */, Windows::UI::Xaml::RoutedEventArgs const& /* e */)
{

```

```
        Splitter().IsPaneOpen(!Splitter().IsPaneOpen());  
    }
```

DisplayChangedFormats

In C#, **DisplayChangedFormats** is a private method belonging to the **MainPage** class, and it's defined in `SampleConfiguration.cs`.

C#

```
private void DisplayChangedFormats()  
{  
    string output = "Clipboard content has changed!" + Environment.NewLine;  
    output += BuildClipboardFormatsOutputString();  
    NotifyUser(output, NotifyType.StatusMessage);  
}
```

In C++/WinRT, we'll make it a private static field of **SampleState** (it doesn't access any instance members), in the `SampleConfiguration.h` and `SampleConfiguration.cpp` files. The C# code for this method doesn't use **System.Text.StringBuilder**; but it does enough string formatting that for the C++/WinRT version this is another good place to use `std::wostringstream`.

Instead of the static `System.Environment.NewLine` property, which is used in the C# code, we'll insert the standard C++ `std::endl` (a newline character) into the output stream.

C++/WinRT

```
// SampleConfiguration.h  
...  
private:  
    static void DisplayChangedFormats();  
...  
  
// SampleConfiguration.cpp  
void SampleState::DisplayChangedFormats()  
{  
    std::wostringstream output;  
    output << L"Clipboard content has changed!" << std::endl;  
    output << BuildClipboardFormatsOutputString().c_str();  
    MainPage::Current().NotifyUser(output.str(), NotifyType::StatusMessage);  
}
```

There is a small inefficiency in the design of the C++/WinRT version above. First, we create a `std::wostringstream`. But we also call the **BuildClipboardFormatsOutputString**

method (which we ported earlier). That method creates its own `std::wstringstream`. And it turns its stream into a `winrt::hstring` and returns that. We call the `hstring::c_str` function to turn that returned `hstring` back into a C-style string, and then we insert that into our stream. It would be more efficient to create just one `std::wstringstream`, and pass (a reference to) that around, so that methods can insert strings into it directly.

That's what we do in the C++/WinRT version of the [Clipboard sample](#) source code (in the ZIP that you downloaded). In that source code, there's a new private static method named `SampleState::AddClipboardFormatsOutputString`, which takes and operates on a reference to an output stream. And then the methods

`SampleState::DisplayChangedFormats` and

`SampleState::BuildClipboardFormatsOutputString` are refactored to call that new method. It's functionally equivalent to the code listings in this topic, but it's more efficient.

Footer_Click

`Footer_Click` is an asynchronous event handler belonging to the C# `MainPage` class, and it's defined in `MainPage.xaml.cs`. The code listing below is functionally equivalent to the method in the source code that you downloaded. But here I've unpacked it from one line to four, to make it easier to see what it's doing, and consequently how we should port it.

C#

```
async void Footer_Click(object sender, RoutedEventArgs e)
{
    var hyperlinkButton = (HyperlinkButton)sender;
    string tagUrl = hyperlinkButton.Tag.ToString();
    Uri uri = new Uri(tagUrl);
    await Windows.System.Launcher.LaunchUriAsync(uri);
}
```

While, technically, the method is asynchronous, it doesn't do anything after the `await`, so it doesn't need the `await` (nor the `async` keyword). It probably uses them in order to avoid the IntelliSense message in Visual Studio.

The equivalent C++/WinRT method will also be asynchronous (because it calls [Launcher.LaunchUriAsync](#)). But it doesn't need to `co_await`, nor to return an asynchronous object. For info about `co_await` and asynchronous objects, see [Concurrency and asynchronous operations with C++/WinRT](#).

Now let's talk about what the method is doing. Because this is an event handler for the **Click** event of a **HyperlinkButton**, the object named *sender* is actually a **HyperlinkButton**. So the type conversion is safe (we could alternatively have expressed this conversion as `sender as HyperlinkButton`). Next, we retrieve the value of the **Tag** property (if you look at the XAML markup in the C# project, you'll see that this is set to a string representing a web url). Although the **FrameworkElement.Tag** property (**HyperlinkButton** is a **FrameworkElement**) is of type **object**, in C# we can stringify that with **Object.ToString**. From the resulting string, we construct a **Uri** object. And finally (with the help of the Shell) we launch a browser and navigate to the url.

Here's the method ported to C++/WinRT (again, expanded for clarity), after which is a description of the details.

C++/WinRT

```
// pch.h
...
#include "winrt/Windows.System.h"
...

// MainPage.h
...
    void Footer_Click(Windows::Foundation::IIinspectable const& sender,
Windows::UI::Xaml::RoutedEventArgs const& e);
private:
...

// MainPage.cpp
...
using namespace winrt::Windows::Foundation;
using namespace winrt::Windows::UI::Xaml::Controls;
...
void MainPage::Footer_Click(Windows::Foundation::IIinspectable const& sender,
Windows::UI::Xaml::RoutedEventArgs const& e)
{
    auto hyperlinkButton{ sender.as<HyperlinkButton>() };
    hstring tagUrl{ winrt::unbox_value<hstring>(hyperlinkButton.Tag()) };
    Uri uri{ tagUrl };
    Windows::System::Launcher::LaunchUriAsync(uri);
}
```

As always, we make the event handler **public**. We use the **as** function on the *sender* object to convert it to **HyperlinkButton**. In C++/WinRT, the **Tag** property is an **IIinspectable** (the equivalent of **Object**). But there's no **Tostring** on **IIinspectable**. Instead, we have to unbox the **IIinspectable** to a scalar value (a string, in this case). Again, for more info on boxing and unboxing, see [Boxing and unboxing values to IIinspectable](#).

The last two lines repeat porting patterns we've seen before, and they pretty much echo the C# version.

HandleClipboardChanged

There's nothing new involved in porting this method. You can compare the C# and C++/WinRT versions in the ZIP of the [Clipboard sample](#) source code that you downloaded.

OnClipboardChanged and OnWindowActivated

So far we have only empty stubs for these two event handlers. But porting them is straightforward, and it doesn't raise anything new to discuss.

ScenarioControl_SelectionChanged

This is another private event handler belonging to the C# **MainPage** class, and defined in `MainPage.xaml.cs`. In C++/WinRT, we'll make it public, and implement it in `MainPage.h` and `MainPage.cpp`.

For this method, we'll need **MainPage::navigating**, which is a private Boolean field, initialized to `false`. And you'll need a **Frame** in `MainPage.xaml`, named *ScenarioFrame*. But, apart from those details, porting this method reveals no new techniques.

If, instead of porting by hand, you're copying code from the C++/WinRT version in the ZIP of the [Clipboard sample](#) source code that you downloaded, then you'll see a **MainPage::NavigateTo** being used there. For now, just refactor the contents of **NavigateTo** into **ScenarioControl_SelectionChanged**.

UpdateStatus

We have only a stub so far for **MainPage.UpdateStatus**. Porting its implementation, again, covers largely old ground. One new point to note is that while in C# we can compare a **string** to **String.Empty**, In C++/WinRT we instead call the [winrt::hstring::empty](#) function. Another is that `nullptr` is the standard C++ equivalent of C#'s `null`.

You can perform the rest of the port with techniques we've already covered. Here's a list of the kinds of things you'll need to do before the ported version of this method will compile.

- To `MainPage.xaml`, add a **Border** named *StatusBorder*.
- To `MainPage.xaml`, add a **TextBlock** named *StatusBlock*.
- To `MainPage.xaml`, add a **StackPanel** named *StatusPanel*.
- To `pch.h`, add `#include "winrt/Windows.UI.Xaml.Media.h"`.
- To `pch.h`, add `#include "winrt/Windows.UI.Xaml.Automation.Peers.h"`.
- To `MainPage.cpp` add `using namespace winrt::Windows::UI::Xaml::Media;`.
- To `MainPage.cpp` add `using namespace winrt::Windows::UI::Xaml::Automation::Peers;`.

Copy the XAML and styles necessary to finish up porting MainPage

For XAML, the ideal case is that you can use *the same* XAML markup across a C# and a C++/WinRT project. And the Clipboard sample is one of those cases.

In its `Styles.xaml` file, the Clipboard sample has a XAML **ResourceDictionary** of styles, which are applied to the buttons, menus, and other UI elements across the UI of the application. The `Styles.xaml` page is merged into `App.xaml`. And then there's the standard `MainPage.xaml` starting point for the UI, which we've already seen briefly. We can now re-use those three `.xaml` files, unchanged, in the C++/WinRT version of the project.

As with asset files, you can choose to reference the same, shared XAML files from multiple versions of your application. In this walkthrough, just for the sake of simplicity, we'll copy files into the C++/WinRT project and add them that way.

Navigate to the `\Clipboard_sample\SharedContent\xaml` folder, select and copy `App.xaml` and `MainPage.xaml`, and then paste those two files into the `\Clipboard\Clipboard` folder in your C++/WinRT project, choosing to replace files when prompted.

In the C++/WinRT project in Visual Studio, click **Show All Files** to toggle it on. Now add a new folder, immediately under the project node, and name it `Styles`. In File Explorer, navigate to the `\Clipboard_sample\SharedContent\xaml` folder, select and copy `Styles.xaml`, and paste it into the `\Clipboard\Clipboard\Styles` folder that you just created. Back in Solution Explorer in the C++/WinRT project, right-click the `Styles` folder > **Add > Existing item...** and navigate to `\Clipboard\Clipboard\Styles`. In the file picker, select `Styles` and click **Add**.

Add a new folder to the C++/WinRT project, immediately under the project node, and named `Styles`. Navigate to the `\Clipboard_sample\SharedContent\xaml` folder, select

and copy `Styles.xaml`, and paste it into the `\Clipboard\Clipboard\Styles` folder in your C++/WinRT project. Right-click the `Styles` folder (in Solution Explorer in the C++/WinRT project) > **Add** > **Existing item...** and navigate to `\Clipboard\Clipboard\Styles`. In the file picker, select `Styles` and click **Add**.

Click **Show All Files** again to toggle it off.

We've now finished porting **MainPage**, and if you've been following along with the steps then your C++/WinRT project will now build and run.

Consolidate your `.idl` files

In addition to the standard `MainPage.xaml` starting point for the UI, the Clipboard sample has five other scenario-specific XAML pages, together with their corresponding code-behind files. We'll be re-using the actual XAML markup of all of these pages, unchanged, in the C++/WinRT version of the project. And we'll look at how to port the code-behind in the next few major sections. But before that, let's talk about IDL.

There's value in consolidating the IDL for your runtime classes into a single IDL file. To learn about that value, see [Factoring runtime classes into Midl files \(.idl\)](#). So next we'll consolidate the contents of `CopyFiles.idl`, `CopyImage.idl`, `CopyText.idl`, `HistoryAndRoaming.idl`, and `OtherScenarios.idl` by moving that IDL into a single file named `Project.idl` (and then deleting the original files).

While we're doing that, let's also remove the auto-generated dummy property (`Int32 MyProperty`; , and its implementation) from each of those five XAML page types.

First, add a new **Midl File (.idl)** item to the C++/WinRT project. Name it `Project.idl`. Replace the entire contents of `Project.idl` with the following code.

idl

```
// Project.idl
namespace SDKTemplate
{
    [default_interface]
    runtimeclass CopyFiles : Windows.UI.Xaml.Controls.Page
    {
        CopyFiles();
    }

    [default_interface]
    runtimeclass CopyImage : Windows.UI.Xaml.Controls.Page
    {
        CopyImage();
    }
}
```



```

    }

    [default_interface]
    runtimeclass CopyText : Windows.UI.Xaml.Controls.Page
    {
        CopyText();
    }

    [default_interface]
    runtimeclass HistoryAndRoaming : Windows.UI.Xaml.Controls.Page
    {
        HistoryAndRoaming();
    }

    [default_interface]
    runtimeclass OtherScenarios : Windows.UI.Xaml.Controls.Page
    {
        OtherScenarios();
    }
}

```

As you can see, that's just a copy of the contents of the individual `.idl` files, all inside one namespace, and with `MyProperty` removed from each runtime class.

In Solution Explorer in Visual Studio, multiple-select all of the original IDL files (`CopyFiles.idl`, `CopyImage.idl`, `CopyText.idl`, `HistoryAndRoaming.idl`, and `OtherScenarios.idl`) and **Edit > Remove** them (choose **Delete** in the dialog).

Finally—and to complete the removal of `MyProperty`—in the `.h` and `.cpp` files for each of those same five XAML page types, delete the declarations and definitions of the `int32_t MyProperty()` accessor and `void MyProperty(int32_t)` mutator functions.

Incidentally, it's always a good idea to have the name of your XAML files match the name of the class that they represent. For example, if you have `x:Class="MyNamespace.MyPage"` in a XAML markup file, then that file should be named `MyPage.xaml`. While this isn't a technical requirement, not having to juggle different names for the same artifact will make your project more understandable and maintainable, and easier to work with.

CopyFiles

In the C# project, the **CopyFiles** XAML page type is implemented in the `CopyFiles.xaml` and `CopyFiles.xaml.cs` source code files. Let's take a look at each of the members of **CopyFiles** in turn.

rootPage

This is a private field.

C#

```
// CopyFiles.xaml.cs
...
public sealed partial class CopyFiles : Page
{
    MainPage rootPage = MainPage.Current;
    ...
}
...
```

In C++/WinRT, we can define and initialize it like this.

C++/WinRT

```
// CopyFiles.h
...
struct CopyFiles : CopyFilesT<CopyFiles>
{
    ...
private:
    SDKTemplate::MainPage rootPage{ MainPage::Current() };
};
...
```

Again (just like with **MainPage::current**), **CopyFiles::rootPage** is declared as being of type **SDKTemplate::MainPage**, which is the projected type, and not the implementation type.

CopyFiles (the constructor)

In the C++/WinRT project, the **CopyFiles** type already has a constructor containing the code we want (it just calls **InitializeComponent**).

CopyButton_Click

The C# **CopyButton_Click** method is an event handler, and from the `async` keyword in its signature we can tell that the method does asynchronous work. In C++/WinRT, we implement an asynchronous method as a *coroutine*. For an introduction to concurrency in C++/WinRT, together with a description of what a *coroutine* is, see [Concurrency and asynchronous operations with C++/WinRT](#).

It's common to want to schedule further work after a coroutine completes, and for such cases the coroutine would return some asynchronous object type that can be awaited, and that optionally reports progress. But those considerations typically don't apply to an event handler. So when you have an event handler that performs asynchronous operations, you can implement that as a coroutine that returns `winrt::fire_and_forget`. For more info, see [Fire and forget](#).

Although the idea of a fire-and-forget coroutine is that you don't care when it completes, work is still continuing (or is suspended, awaiting resumption) in the background. You can see from the C# implementation that `CopyButton_Click` depends on the `this` pointer (it accesses the instance data member `rootPage`). So we must be sure that the `this` pointer (a pointer to a `CopyFiles` object) outlives the `CopyButton_Click` coroutine. In a situation like this sample application, where the user navigates between UI pages, we can't directly control the lifetime of those pages. Should the `CopyFiles` page be destroyed (by navigating away from it) while `CopyButton_Click` is still in flight on a background thread, it won't be safe to access `rootPage`. To make the coroutine correct, it needs to obtain a strong reference to the `this` pointer, and keep that reference for the duration of the coroutine. For more info, see [Strong and weak references in C++/WinRT](#).

If you look in the C++/WinRT version of the sample, at `CopyFiles::CopyButton_Click`, you'll see that it's done with a simple declaration on the stack.

C++/WinRT

```
fire_and_forget CopyFiles::CopyButton_Click(IInspectable const&,
RoutedEventArgs const&)
{
    auto lifetime{ get_strong() };
    ...
}
```

Let's look at the other aspects of the ported code that are noteworthy.

In the code, we instantiate a `FileOpenPicker` object, and two lines later we access that object's `FileTypeFilter` property. The return type of that property implements an `IVector` of strings. And on that `IVector`, we call the `IVector<T>.ReplaceAll(T[])` method. The interesting aspect is the value that we're passing to that method, where an array is expected. Here's the line of code.

C++/WinRT

```
filePicker.FileTypeFilter().ReplaceAll({ L "*" });
```

The value that we're passing (`{ L"" }`) is a standard C++ *initializer list*. It contains a single object, in this case, but an initializer list can contain any number of comma-separated objects. The pieces of C++/WinRT that allow you the convenience of passing an initializer list to a method such as this are explained in [Standard initializer lists](#).

We port the C# `await` keyword to `co_await` in C++/WinRT. Here's the example from the code.

C++/WinRT

```
auto storageItems{ co_await filePicker.PickMultipleFilesAsync() };
```

Next, consider this line of C# code.

C#

```
dataPackage.SetStorageItems(storageItems);
```

C# is able to implicitly convert the `IReadOnlyList<StorageFile>` represented by `storageItems` into the `IEnumerable<IStorageItem>` expected by [DataPackage.SetStorageItems](#). But in C++/WinRT we need to explicitly convert from `IVectorView<StorageFile>` to `Iterable<IStorageItem>`. And so we have another example of the `as` function in action.

C++/WinRT

```
dataPackage.SetStorageItems(storageItems.as<IVectorView<IStorageItem>>());
```

Where we use the `null` keyword in C# (for example, `Clipboard.SetContentWithOptions(dataPackage, null)`), we use `nullptr` in C++/WinRT (for example, `Clipboard::SetContentWithOptions(dataPackage, nullptr)`).

PasteButton_Click

This is another event handler in the form of a fire-and-forget coroutine. Let's look at the aspects of the ported code that are noteworthy.

In the C# version of the sample, we catch exceptions with `catch (Exception ex)`. In the ported C++/WinRT code, you'll see the expression `catch (winrt::hresult_error const& ex)`. For more info about [winrt::hresult_error](#) and how to work with it, see [Error handling with C++/WinRT](#).

An example of testing whether a C# object is `null` or not is `if (storageItems != null)`. In C++/WinRT, we can rely on a conversion operator to `bool`, which does the test against `nullptr` internally.

Here's a slightly simplified version of a fragment of code from the ported C++/WinRT version of the sample.

C++/WinRT

```
std::wostream output;  
output <<  
std::wstring_view(ApplicationData::Current().LocalFolder().Path());
```

Constructing a `std::wstring_view` from a `winrt::hstring` like that illustrates an alternative to calling the `hstring::c_str` function (to turn the `winrt::hstring` into a C-style string). This alternative works thanks to `hstring`'s [conversion operator to `std::wstring\_view`](#).

Consider this fragment of C#.

C#

```
var file = storageItem as StorageFile;  
if (file != null)  
...
```

To port the C# `as` keyword to C++/WinRT, so far we've seen the `as` function used a couple of times. That function throws an exception if the type conversion fails. But if we want the conversion to return `nullptr` if it fails (so that we can handle that condition in the code), then we instead use the `try_as` function.

C++/WinRT

```
auto file{ storageItem.try_as<StorageFile>() };  
if (file)  
...
```

Copy the XAML necessary to finish up porting CopyFiles

You can now select the entire contents of the `CopyFiles.xaml` file from the `shared` folder of the original sample source code download, and paste that into the `CopyFiles.xaml` file in the C++/WinRT project (replacing the existing contents of that file in the C++/WinRT project).

Finally, edit `CopyFiles.h` and `.cpp` and delete the dummy `ClickHandler` function, since we just overwrote the corresponding XAML markup.

We've now finished porting `CopyFiles`, and if you've been following along with the steps then your C++/WinRT project will now build and run, and the `CopyFiles` scenario will be functional.

CopyImage

To port the `CopyImage` XAML page type, you follow the same process as for `CopyFiles`. While porting `CopyImage`, you'll encounter the use of the C# *using statement*, which ensures that objects that implement the `IDisposable` interface are disposed correctly.

C#

```
if (imageReceived != null)
{
    using (var imageStream = await imageReceived.OpenReadAsync())
    {
        ... // Pass imageStream to other APIs, and do other work.
    }
}
```

The equivalent interface in C++/WinRT is `IClosable`, with its single `Close` method. Here's the C++/WinRT equivalent of the C# code above.

C++/WinRT

```
if (imageReceived)
{
    auto imageStream{ co_await imageReceived.OpenReadAsync() };
    ... // Pass imageStream to other APIs, and do other work.
    imageStream.Close();
}
```

C++/WinRT objects implement `IClosable` primarily for the benefit of languages that lack deterministic finalization. C++/WinRT has deterministic finalization, and so we often don't need to call `IClosable::Close` when we're writing C++/WinRT. But there are times when it's good to call it, and this is one of those times. Here, the *imageStream* identifier is a reference-counted wrapper around an underlying Windows Runtime object (in this case, an object that implements `IRandomAccessStreamWithContentType`). Although we can determine that the finalizer of *imageStream* (its destructor) will run at the end of the enclosing scope (the curly brackets), we can't be certain that that finalizer will call `Close`. That's because we passed *imageStream* to other APIs, and they might still be

contributing to the reference count of the underlying Windows Runtime object. So this is a case where it's a good idea to call **Close** explicitly. For more info, see [Do I need to call IClosable::Close on runtime classes that I consume?](#).

Next, consider the C# expression `(uint)(imageDecoder.OrientedPixelWidth * 0.5)`, which you'll find in the **OnDeferredImageRequestedHandler** event handler. That expression multiplies a `uint` by a `double`, resulting in a `double`. It then casts that to a `uint`. In C++/WinRT, we *could* use a similar-looking C-style cast `((uint32_t)(imageDecoder.OrientedPixelWidth() * 0.5))`, but it's preferable to make it clear exactly what kind of cast we intend, and in this case we would do that with `static_cast<uint32_t>(imageDecoder.OrientedPixelWidth() * 0.5)`.

The C# version of **CopyImage.OnDeferredImageRequestedHandler** has a `finally` clause, but not a `catch` clause. We went just a little bit further in the C++/WinRT version, and implemented a `catch` clause so that we can report whether or not the delayed rendering was successful.

Porting the remainder of this XAML page doesn't yield anything new to discuss. Remember to delete the dummy **ClickHandler** function. And, just like with **CopyFiles**, the last step in the port is to select the entire contents of `CopyImage.xaml`, and paste it into the same file in the C++/WinRT project.

CopyText

You can port `CopyText.xaml` and `CopyText.xaml.cs` using techniques we've already covered.

HistoryAndRoaming

There are some points of interest that arise while porting the **HistoryAndRoaming** XAML page type.

First, take a look at the C# source code, and follow the flow of control from **OnNavigatedTo** through the **OnHistoryEnabledChanged** event handler, and finally to the asynchronous function **CheckHistoryAndRoaming** (which is not awaited, so it's essentially fire and forget). Because **CheckHistoryAndRoaming** is asynchronous, we'll need to be careful in C++/WinRT about the lifetime of the `this` pointer. You can see the outcome if you look at the implementation in the `HistoryAndRoaming.cpp` source code file. First, when we attach delegates to the **Clipboard::HistoryEnabledChanged** and **Clipboard::RoamingEnabledChanged** events, we take only a weak reference to the

HistoryAndRoaming page object. We do that by creating the delegate with a dependency on the value returned from `winrt::get_weak`, instead of a dependency on the `this` pointer. Which means that the delegate itself, which eventually calls into asynchronous code, doesn't keep the **HistoryAndRoaming** page alive, should we navigate away from it.

And second, when we do finally reach our fire-and-forget **CheckHistoryAndRoaming** coroutine, the first thing we do is to take a strong reference to `this` to guarantee that the **HistoryAndRoaming** page lives at least until the coroutine finally completes. For more info about both of the aspects just described, see [Strong and weak references in C++/WinRT](#).

We find another point of interest while porting **CheckHistoryAndRoaming**. It contains code to update the UI; so we need to be certain that we're doing that on the main UI thread. The thread that initially calls into an event handler is the main UI thread. But typically, an asynchronous method can execute and/or resume on any arbitrary thread. In C#, the solution is to call `CoreDispatcher.RunAsync`, and update the UI from within the lambda function. In C++/WinRT, we can use the `winrt::resume_foreground` function together with the `this` pointer's `Dispatcher` to suspend the coroutine and immediately resume on the main UI thread.

The relevant expression is `co_await winrt::resume_foreground(Dispatcher());`. Alternatively, although with less clarity, we could express that simply as `co_await Dispatcher();`. The shorter version is achieved courtesy of a conversion operator supplied by C++/WinRT.

Porting the remainder of this XAML page doesn't yield anything new to discuss. Remember to delete the dummy **ClickHandler** function, and to copy over the XAML markup.

OtherScenarios

You can port `OtherScenarios.xaml` and `OtherScenarios.xaml.cs` using techniques we've already covered.

Conclusion

Hopefully this walkthrough has armed you with sufficient porting info and techniques that you can now go ahead and port your own C# applications to C++/WinRT. By way of a refresher, you can continue to refer back to the *before* (C#) and *after* (C++/WinRT)

versions of the source code in the [Clipboard sample](#), and compare them side by side to see the correspondence.

Related topics

- [Move to C++/WinRT from C#](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Move to C++/WinRT from C#

Article • 10/27/2022

Tip

If you've read this topic before, and you're returning to it with a particular task in mind, then you can jump to the [Find content based on the task you're performing](#) section of this topic.

This topic comprehensively catalogs the technical details involved in porting the source code in a [C#](#) project to its equivalent in [C++/WinRT](#).

For a case study of porting one of the Universal Windows Platform (UWP) app samples, see the companion topic [Porting the Clipboard sample to C++/WinRT from C#](#). You can gain porting practice and experience by following along with that walkthrough, and porting the sample for yourself as you go.

How to prepare, and what to expect

The case study [Porting the Clipboard sample to C++/WinRT from C#](#) illustrates examples of the kinds of software design decisions that you'll make while porting a project to C++/WinRT. So, it's a good idea to prepare for porting by gaining a solid understanding of how the existing code works. That way, you'll get a good overview of the app's functionality, and the code's structure, and then the decisions that you make will always take you forward, and in the right direction.

In terms of what kinds of porting changes to expect, you could group them into four categories.

- **Port the language projection.** The Windows Runtime (WinRT) is *projected* into various programming languages. Each of those language projections is designed to feel idiomatic to the programming language in question. For C#, some Windows Runtime types are projected as .NET types. So for example you'll be translating `System.Collections.Generic.IReadOnlyList<T>` back to `Windows.Foundation.Collections.IVectorView<T>`. Also in C#, some Windows Runtime operations are projected as convenient C# language features. An example is that in C# you use the `+=` operator syntax to register an event-handling delegate. So you'll be translating language features such as that back to the fundamental operation that's being performed (event registration, in this example).
- **Port language syntax.** Many of these changes are simple mechanical transforms, replacing one symbol for another. For example, changing dot (`.`) to double-colon (`::`).
- **Port language procedure.** Some of these can be simple, repetitive changes (such as `myObject.MyProperty` to `myObject.MyProperty()`). Others need deeper changes (for example, porting a procedure that involves the use of `System.Text.StringBuilder` to one that involves the use of `std::wostream`).
- **Porting-related tasks that are specific to C++/WinRT.** Certain details of the Windows Runtime are taken care of implicitly by C#, behind the scenes. Those details are done explicitly in C++/WinRT. An example is that you use an `.idl` file to define your runtime classes.

After the task-based index that follows, the rest of the sections in this topic are structured according to the taxonomy above.

Find content based on the task you're performing

Task	Content
Author a Windows Runtime component (WRC)	Certain functionality can be achieved (or certain APIs called) only with C++. You can factor that functionality into a C++/WinRT WRC, and then consume the WRC from (for example) a C# app. See Windows Runtime components with C++/WinRT and If you're authoring a runtime class in a Windows Runtime component .
Port an async method	It's a good idea for the first line of an asynchronous method in a C++/WinRT runtime class to be <code>auto lifetime = get_strong();</code> (see Safely accessing the <i>this</i> pointer in a class-member coroutine). Porting from <code>Task</code>, see Async action. Porting from <code>Task<T></code>, see Async operation. Porting from <code>async void</code>, see Fire-and-forget method.
Port a class	First, determine whether the class needs to be a runtime class, or whether it can be an ordinary class. To help you decide that, see the very beginning of Author APIs with C++/WinRT . Then, see the next three rows below.
Port a runtime class	A class that shares functionality outside of the C++ app, or a class that's used in XAML data binding. See If you're authoring a runtime class in a Windows Runtime component, or If you're authoring a runtime class to be referenced in your XAML UI. Those links describe this in more detail, but a runtime class must be declared in IDL. If your project already contains an IDL file (for example, <code>Project.idl</code>), then we recommend that you declare any new runtime class in that file. In IDL, declare any methods and data members that will be used outside your app, or that will be used in XAML. After updating the IDL file, rebuild, and look at the generated stub files (<code>.h</code> and <code>.cpp</code>) in your project's <code>Generated Files</code> folder (In Solution Explorer, with the project node selected, make sure Show All Files is toggled on). Compare the stub files with the files already in your project, adding files or adding/updating function signatures as necessary. Stub file syntax is always correct, so we recommend that you use it in order to minimize build errors. Once the stubs in your project match those in the stub files, you can go ahead and implement them by porting the C# code over.
Port an ordinary class	See If you're <i>not</i> authoring a runtime class .
Author IDL	Introduction to Microsoft Interface Definition Language 3.0 If you're authoring a runtime class to be referenced in your XAML UI Consuming objects from XAML markup Define your runtime classes in IDL
Port a collection	Collections with C++/WinRT Making a data source available to XAML markup Associative container Vector member access
Port an event	Event handler delegate as class member Revoke event handler delegate
Port a method	From C#: <code>private async void SampleButton_Tapped(object sender, Windows.UI.Xaml.Input.TappedRoutedEventArgs e) { ... }</code> To the C++/WinRT <code>.h</code> file: <code>fire_and_forget SampleButton_Tapped(IInspectable const&, RoutedEventArgs const&);</code> To the C++/WinRT <code>.cpp</code> file: <code>fire_and_forget OcrFileImage::SampleButton_Tapped(IInspectable const&, RoutedEventArgs const&) {...}</code>
Port strings	String handling in C++/WinRT ToString

Task	Content
	String-building Boxing and unboxing a string
Type conversion (type casting)	C#: <code>o.ToString()</code> C++/WinRT: <code>to_hstring(static_cast<int>(o))</code> Also see ToString. C#: <code>(Value)o</code> C++/WinRT: <code>unbox_value<Value>(o)</code> Throws if unboxing fails. Also see Boxing and unboxing. C#: <code>o as Value? ?? fallback</code> C++/WinRT: <code>unbox_value_or<Value>(o, fallback)</code> Returns fallback if unboxing fails. Also see Boxing and unboxing. C#: <code>(Class)o</code> C++/WinRT: <code>o.as<Class>()</code> Throws if conversion fails. C#: <code>o as Class</code> C++/WinRT: <code>o.try_as<Class>()</code> Returns null if conversion fails.

Changes that involve the language projection

 Expand table

Category	C#	C++/WinRT	See also
Untyped object	<code>object</code> , or System.Object	Windows::Foundation::IInspectable	Porting the EnableClipboardContentChangedNotifications method
Projection namespaces	<code>using System;</code>	<code>using namespace</code> <code>Windows::Foundation;</code>	
	<code>using</code> <code>System.Collections.Generic;</code>	<code>using namespace</code> <code>Windows::Foundation::Collections;</code>	
Size of a collection	<code>collection.Count</code>	<code>collection.Size()</code>	Porting the BuildClipboardFormatsOutputString method
Typical collection type	IList<T> , and Add to add an element.	IVector<T> , and Append to add an element. If you use a <code>std::vector</code> anywhere, then push_back to add an element.	
Read-only collection type	IReadOnlyList<T>	IVectorView<T>	Porting the BuildClipboardFormatsOutputString method
Event handler delegate as class member	<code>myObject.EventName +=</code> <code>Handler;</code>	<code>token = myObject.EventName({</code> <code>get_weak(), &Class::Handler });</code>	Porting the EnableClipboardContentChangedNotifications method

Category	C#	C++/WinRT	See also
Revoke event handler delegate	<code>myObject.EventName -= Handler;</code>	<code>myObject.EventName(token);</code>	Porting the EnableClipboardContentChangedNotifications method
Associative container	IDictionary<K, V>	IMap<K, V>	
Vector member access	<code>x = v[i];</code> <code>v[i] = x;</code>	<code>x = v.GetAt(i);</code> <code>v.SetAt(i, x);</code>	

Register/revoke an event handler

In C++/WinRT, you have several syntactic options to register/revoke an event handler delegate, as described in [Handle events by using delegates in C++/WinRT](#). Also see [Porting the EnableClipboardContentChangedNotifications method](#).

Sometimes, for example when an event recipient (an object handling an event) is about to be destroyed, you'll want to revoke an event handler so that the event source (the object raising the event) doesn't call into a destroyed object. See [Revoke a registered delegate](#). In cases like that, create an **event_token** member variable for your event handlers. For an example, see [Porting the EnableClipboardContentChangedNotifications method](#).

You can also register an event handler in XAML markup.

XAML

```
<Button x:Name="OpenButton" Click="OpenButton_Click" />
```

In C#, your **OpenButton_Click** method can be private, and XAML will still be able to connect it to the [ButtonBase.Click](#) event raised by *OpenButton*.

In C++/WinRT, your **OpenButton_Click** method must be public in your [implementation type](#) if you want to register it in XAML markup. If you register an event handler only in imperative code, then the event handler doesn't need to be public.

C++/WinRT

```
namespace winrt::MyProject::implementation
{
    struct MyPage : MyPageT<MyPage>
    {
        void OpenButton_Click(
            winrt::Windows::Foundation::IIinspectable const& sender,
            winrt::Windows::UI::Xaml::RoutedEventArgs const& args);
    }
};
```

Alternatively, you can make the registering XAML page a friend of your implementation type, and **OpenButton_Click** private.

C++/WinRT

```
namespace winrt::MyProject::implementation
{
    struct MyPage : MyPageT<MyPage>
    {
    private:
        friend MyPageT;
        void OpenButton_Click(
            winrt::Windows::Foundation::IIinspectable const& sender,
            winrt::Windows::UI::Xaml::RoutedEventArgs const& args);
    }
};
```

One final scenario is where the C# project that you're porting *binds* to the event handler from markup (for more background on that scenario, see [Functions in x:Bind](#)).

```
XAML

<Button x:Name="OpenButton" Click="{x:Bind OpenButton_Click}" />
```

You could just change that markup to the more simple `Click="OpenButton_Click"`. Or, if you prefer, you can keep that markup as it is. All you have to do to support it is to declare the event handler in IDL.

```
idl

void OpenButton_Click(Object sender, Windows.UI.Xaml.RoutedEventArgs e);
```

Note

Declare the function as `void` even if you *implement* it as [Fire and forget](#).

Changes that involve the language syntax

[Expand table](#)

Category	C#	C++/WinRT	See also
Access modifiers	<code>public</code> \<member\>	<code>public:</code> \<member\>	Porting the Button_Click method
Access a data member	<code>this.variable</code>	<code>this->variable</code>	
Async action	<code>async Task ...</code>	<code>IAsyncAction ...</code>	IAsyncAction interface, Concurrency and asynchronous operations with C++/WinRT
Async operation	<code>async Task<T> ...</code>	<code>IAsyncOperation<T> ...</code>	IAsyncOperation interface, Concurrency and asynchronous operations with C++/WinRT
Fire-and-forget method (implies async)	<code>async void ...</code>	<code>winrt::fire_and_forget ...</code>	Porting the CopyButton_Click method, Fire and forget

Category	C#	C++/WinRT	See also
Access an enumerated constant	<code>E.Value</code>	<code>E::Value</code>	Porting the <code>DisplayChangedFormats</code> method
Cooperatively wait	<code>await ...</code>	<code>co_await ...</code>	Porting the <code>CopyButton_Click</code> method
Collection of projected types as a private field	<code>private</code> <code>List<MyRuntimeClass></code> <code>myRuntimeClasses = new</code> <code>List<MyRuntimeClass>();</code>	<code>std::vector</code> <code><MyNamespace::MyRuntimeClass></code> <code>m_myRuntimeClasses;</code>	
GUID construction	<code>private static readonly</code> <code>Guid myGuid = new</code> <code>Guid("C380465D-2271-</code> <code>428C-9B83-</code> <code>ECEA3B4A85C1");</code>	<code>winrt::guid myGuid{ 0xC380465D,</code> <code>0x2271, 0x428C, { 0x9B, 0x83,</code> <code>0xEC, 0xEA, 0x3B, 0x4A, 0x85,</code> <code>0xC1} };</code>	
Namespace separator	<code>A.B.T</code>	<code>A::B::T</code>	
Null	<code>null</code>	<code>nullptr</code>	Porting the <code>UpdateStatus</code> method
Obtain a type object	<code>typeof(MyType)</code>	<code>winrt::xaml_typename<MyType>()</code>	Porting the <code>Scenarios</code> property
Parameter declaration for a method	<code>MyType</code>	<code>MyType const&</code>	Parameter-passing
Parameter declaration for an async method	<code>MyType</code>	<code>MyType</code>	Parameter-passing
Call a static method	<code>T.Method()</code>	<code>T::Method()</code>	
Strings	<code>string</code> , or <code>System.String</code>	winrt::hstring	String handling in C++/WinRT
String literal	<code>"a string literal"</code>	<code>L"a string literal"</code>	Porting the constructor, <code>Current</code>, and <code>FEATURE_NAME</code>
Inferred (or deduced) type	<code>var</code>	<code>auto</code>	Porting the <code>BuildClipboardFormatsOutputString</code> method
Using-directive	<code>using A.B.C;</code>	<code>using namespace A::B::C;</code>	Porting the constructor, <code>Current</code>, and <code>FEATURE_NAME</code>
Verbatim/raw string literal	<code>@"verbatim string</code> <code>literal"</code>	<code>LR"(raw string literal)"</code>	Porting the <code>DisplayToast</code> method

❗ Note

If a header file doesn't contain a `using namespace` directive for a given namespace, then you'll have to fully-qualify all type names for that namespace; or at least qualify them sufficiently for the compiler to find them. For an example, see [Porting the `DisplayToast` method](#).

Porting classes and members

You'll need to decide, for each C# type, whether to port it to a Windows Runtime type, or to a regular C++ class/struct/enumeration. For more info, and detailed examples illustrating how to make those decisions, see [Porting the Clipboard sample to C++/WinRT from C#](#).

A C# property typically becomes an accessor function, a mutator function, and a backing data member. For more info, and an example, see [Porting the IsClipboardContentChangedEnabled property](#).

For non-static fields, make them data members of your [implementation type](#).

A C# static field becomes a C++/WinRT static accessor and/or mutator function. For more info, and an example, see [Porting the constructor, Current, and FEATURE_NAME](#).

For member functions, again, you'll need to decide for each one whether or not it belongs in the IDL, or whether it's a public or private member function of your implementation type. For more info, and examples of how to decide, see [IDL for the MainPage type](#).

Porting XAML markup, and asset files

In the case of [Porting the Clipboard sample to C++/WinRT from C#](#), we were able to use *the same* XAML markup (including resources) and asset files across the C# and the C++/WinRT project. In some cases, edits to markup will be necessary to achieve that. See [Copy the XAML and styles necessary to finish up porting MainPage](#).

Changes that involve procedures within the language

 Expand table

Category	C#	C++/WinRT	See also
Lifetime management in an async method	N/A	<code>auto lifetime{ get_strong() }; or auto lifetime = get_strong();</code>	Porting the CopyButton_Click method
Disposal	<code>using (var t = v)</code>	<code>auto t{ v }; t.Close(); // or let wrapper destructor do the work</code>	Porting the CopyImage method
Construct object	<code>new MyType(args)</code>	<code>MyType{ args } or MyType(args)</code>	Porting the Scenarios property
Create uninitialized reference	<code>MyType myObject;</code>	<code>MyType myObject{ nullptr }; or MyType myObject = nullptr;</code>	Porting the constructor, Current, and FEATURE_NAME
Construct object into variable with args	<code>var myObject = new MyType(args);</code>	<code>auto myObject{ MyType{ args } }; or auto myObject{ MyType(args) }; or auto myObject = MyType{ args }; or auto myObject = MyType(args); or MyType myObject{ args }; or MyType myObject(args);</code>	Porting the Footer_Click method
Construct object into	<code>var myObject = new T();</code>	<code>MyType myObject;</code>	Porting the BuildClipboardFormatsOutputString

Category	C#	C++/WinRT	See also
variable without args			method
Object initialization shorthand	<pre>var p = new FileOpenPicker{ ViewMode = PickerViewMode.List };</pre>	<pre>FileOpenPicker p; p.ViewMode(PickerViewMode::List);</pre>	
Bulk vector operation	<pre>var p = new FileOpenPicker{ FileTypeFilter = { ".png", ".jpg", ".gif" } };</pre>	<pre>FileOpenPicker p; p.FileTypeFilter().ReplaceAll({ L".png", L".jpg", L".gif" });</pre>	Porting the CopyButton_Click method
Iterate over collection	<pre>foreach (var v in c)</pre>	<pre>for (auto&& v : c)</pre>	Porting the BuildClipboardFormatsOutputString method
Catch an exception	<pre>catch (Exception ex)</pre>	<pre>catch (winrt::hresult_error const& ex)</pre>	Porting the PasteButton_Click method
Exception details	<pre>ex.Message</pre>	<pre>ex.message()</pre>	Porting the PasteButton_Click method
Get a property value	<pre>myObject.MyProperty</pre>	<pre>myObject.MyProperty()</pre>	Porting the NotifyUser method
Set a property value	<pre>myObject.MyProperty = value;</pre>	<pre>myObject.MyProperty(value);</pre>	
Increment a property value	<pre>myObject.MyProperty += v;</pre>	<pre>myObject.MyProperty(thing.Property() + v);</pre> For strings, switch to a builder	
ToString()	<pre>myObject.ToString()</pre>	<pre>winrt::to_hstring(myObject)</pre>	ToString()
Language string to Windows Runtime string	N/A	<pre>winrt::hstring{ s }</pre>	
String- building	<pre>StringBuilder builder; builder.Append(...);</pre>	<pre>std::wstringstream builder; builder << ...;</pre>	String-building
String interpolation	<pre>\${i++}) {s.Title}"</pre>	winrt::to_hstring , and/or winrt::hstring::operator+	Porting the OnNavigatedTo method
Empty string for comparison	System.String.Empty	winrt::hstring::empty	Porting the UpdateStatus method
Create empty string	<pre>var myEmptyString = String.Empty;</pre>	<pre>winrt::hstring myEmptyString{ L"" };</pre>	
Dictionary operations	<pre>map[k] = v; // replaces any existing v = map[k]; // throws if</pre>	<pre>map.Insert(k, v); // replaces any existing v = map.Lookup(k); // throws if not</pre>	

Category	C#	C++/WinRT	See also
	not present <code>map.ContainsKey(k)</code>	present <code>map.HasKey(k)</code>	
Type conversion (throw on failure)	<code>(MyType)v</code>	<code>v.as<MyType>()</code>	Porting the Footer_Click method
Type conversion (null on failure)	<code>v as MyType</code>	<code>v.try_as<MyType>()</code>	Porting the PasteButton_Click method
XAML elements with x:Name are properties	<code>MyNamedElement</code>	<code>MyNamedElement()</code>	Porting the constructor, Current , and FEATURE_NAME
Switch to the UI thread	<code>CoreDispatcher.RunAsync</code>	<code>CoreDispatcher.RunAsync</code> , or winrt::resume_foreground	Porting the NotifyUser method, and Porting the HistoryAndRoaming method
UI element construction in imperative code in a XAML page	See UI element construction	See UI element construction	

The following sections go into more detail regarding some of the items in the table.

UI element construction

These code examples show the construction of a UI element in the imperative code of a XAML page.

C#

```
var myTextBlock = new TextBlock()
{
    Text = "Text",
    Style = (Windows.UI.Xaml.Style)this.Resources["MyTextBlockStyle"]
};
```

C++/WinRT

```
TextBlock myTextBlock;
myTextBlock.Text(L"Text");
myTextBlock.Style(
    winrt::unbox_value<Windows::UI::Xaml::Style>(
        Resources().Lookup(
            winrt::box_value(L"MyTextBlockStyle")
        )
    )
);
```

ToString()

C# types provide the [Object.ToString](#) method.

```
C#  
  
int i = 2;  
var s = i.ToString(); // s is a System.String with value "2".
```

C++/WinRT doesn't directly provide this facility, but you can turn to alternatives.

```
C++/WinRT  
  
int i{ 2 };  
auto s{ std::to_wstring(i) }; // s is a std::wstring with value L"2".
```

C++/WinRT also supports [winrt::to_hstring](#) for a limited number of types. You'll need to add overloads for any additional types you want to stringify.

[Expand table](#)

Language	Stringify int	Stringify enum
C#	<pre>string result = "hello, " + intValue.ToString(); string result = \$"hello, {intValue}";</pre>	<pre>string result = "status: " + status.ToString(); string result = \$"status: {status}";</pre>
C++/WinRT	<pre>hstring result = L"hello, " + to_hstring(intValue);</pre>	<pre>// must define overload (see below) hstring result = L"status: " + to_hstring(status);</pre>

In the case of stringifying an enum, you will need to provide the implementation of [winrt::to_hstring](#).

```
C++/WinRT  
  
namespace winrt  
{  
    hstring to_hstring(StatusEnum status)  
    {  
        switch (status)  
        {  
            case StatusEnum::Success: return L"Success";  
            case StatusEnum::AccessDenied: return L"AccessDenied";  
            case StatusEnum::DisabledByPolicy: return L"DisabledByPolicy";  
            default: return to_hstring(static_cast<int>(status));  
        }  
    }  
}
```

These stringifications are often consumed implicitly by data binding.

```
XAML  
  
<TextBlock>  
You have <Run Text="{Binding FlowerCount}"/> flowers.  
</TextBlock>  
<TextBlock>  
Most recent status is <Run Text="{x:Bind LatestOperation.Status}"/>.  
</TextBlock>
```

These bindings will perform `winrt::to_hstring` of the bound property. In the case of the second example (the `StatusEnum`), you must provide your own overload of `winrt::to_hstring`, otherwise you'll get a compiler error.

Also see [Porting the Footer_Click method](#).

String-building

For string building, C# has a built-in `StringBuilder` type.

 Expand table

Category	C#	C++/WinRT
String-building	<code>StringBuilder builder;</code> <code>builder.Append(...);</code>	<code>std::wstringstream builder;</code> <code>builder << ...;</code>
Append a Windows Runtime string, preserving nulls	<code>builder.Append(s);</code>	<code>builder << std::wstring_view{ s</code> <code>};</code>
Add a newline	<code>builder.Append(Environment.NewLine);</code>	<code>builder << std::endl;</code>
Access the result	<code>s = builder.ToString();</code>	<code>ws = builder.str();</code>

Also see [Porting the BuildClipboardFormatsOutputString method](#), and [Porting the DisplayChangedFormats method](#).

Running code on the main UI thread

This example is taken from the [Barcode scanner sample](#).

When you want to do work on the main UI thread in a C# project, you typically use the `CoreDispatcher.RunAsync` method, like this.

C#

```
private async void Watcher_Added(DeviceWatcher sender, DeviceInformation args)
{
    await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
    {
        // Do work on the main UI thread here.
    });
}
```

It's much simpler to express that in C++/WinRT. Notice that we're accepting parameters by value on the assumption we'll want to access them after the first suspension point (the `co_await`, in this case). For more info, see [Parameter-passing](#).

C++/WinRT

```
winrt::fire_and_forget Watcher_Added(DeviceWatcher sender, winrt::DeviceInformation args)
{
    co_await Dispatcher();
    // Do work on the main UI thread here.
}
```

If you need to do the work at a priority other than the default, then see the [winrt::resume_foreground](#) function, which has an overload that takes a priority. For code examples showing how to await a call to `winrt::resume_foreground`, see [Programming with thread affinity in mind](#).

Porting-related tasks that are specific to C++/WinRT

Define your runtime classes in IDL

See [IDL for the MainPage type](#), and [Consolidate your .idl files](#).

Include the C++/WinRT Windows namespace header files that you need

In C++/WinRT, whenever you want to use a type from a Windows namespaces, you need to include the corresponding C++/WinRT Windows namespace header file. For an example, see [Porting the NotifyUser method](#).

Boxing and unboxing

C# automatically boxes scalars into objects. C++/WinRT requires you to call the [winrt::box_value](#) function explicitly. Both languages require you to unbox explicitly. See [Boxing and unboxing with C++/WinRT](#).

In the tables that follows, we'll use these definitions.

[Expand table](#)

C#	C++/WinRT
<code>int i;</code>	<code>int i;</code>
<code>string s;</code>	<code>winrt::hstring s;</code>
<code>object o;</code>	<code>IInspectable o;</code>

[Expand table](#)

Operation	C#	C++/WinRT
Boxing	<code>o = 1;</code>	<code>o = box_value(1);</code>
	<code>o = "string";</code>	<code>o = box_value(L"string");</code>
Unboxing	<code>i = (int)o;</code>	<code>i = unbox_value<int>(o);</code>
	<code>s = (string)o;</code>	<code>s = unbox_value<winrt::hstring>(o);</code>

C++/CX and C# raise exceptions if you try to unbox a null pointer to a value type. C++/WinRT considers this a programming error, and it crashes. In C++/WinRT, use the [winrt::unbox_value_or](#) function if you want to handle the case where the object is not of the type that you thought it was.

[Expand table](#)

Scenario	C#	C++/WinRT
Unbox a known integer	<code>i = (int)o;</code>	<code>i = unbox_value<int>(o);</code>
If o is null	<code>System.NullReferenceException</code>	Crash
If o is not a boxed int	<code>System.InvalidCastException</code>	Crash
Unbox int, use fallback if null; crash if anything else	<code>i = o != null ? (int)o : fallback;</code>	<code>i = o ? unbox_value<int>(o) : fallback;</code>
Unbox int if possible; use fallback for anything else	<code>i = as int? ?? fallback;</code>	<code>i = unbox_value_or<int>(o, fallback);</code>

For an example, see [Porting the OnNavigatedTo method](#), and [Porting the Footer_Click method](#).

Boxing and unboxing a string

A string is in some ways a value type, and in other ways a reference type. C# and C++/WinRT treat strings differently.

The ABI type **HSTRING** is a pointer to a reference-counted string. But it doesn't derive from **IInspectable**, so it's not technically an *object*. Furthermore, a null **HSTRING** represents the empty string. Boxing of things not derived from **IInspectable** is done by wrapping them inside an **IReference<T>**, and the Windows Runtime provides a standard implementation in the form of the **PropertyValue** object (custom types are reported as **PropertyType::OtherType**).

C# represents a Windows Runtime string as a reference type; while C++/WinRT projects a string as a value type. This means that a boxed null string can have different representations depending how you got there.

[Expand table](#)

Behavior	C#	C++/WinRT
Declarations	<code>object o;</code> <code>string s;</code>	<code>IInspectable o;</code> <code>hstring s;</code>
String type category	Reference type	Value type
null HSTRING projects as	<code>""</code>	<code>hstring{}</code>
Are null and <code>""</code> identical?	No	Yes
Validity of null	<code>s = null;</code> <code>s.Length</code> raises <code>NullReferenceException</code>	<code>s = hstring{};</code> <code>s.size() == 0</code> (valid)
If you assign null string to object	<code>o = (string)null;</code> <code>o == null</code>	<code>o = box_value(hstring{});</code> <code>o != nullptr</code>
If you assign <code>""</code> to object	<code>o = "";</code> <code>o != null</code>	<code>o = box_value(hstring{L""});</code> <code>o != nullptr</code>

Basic boxing and unboxing.

[Expand table](#)

Operation	C#	C++/WinRT
Box a string	<code>o = s;</code> Empty string becomes non-null object.	<code>o = box_value(s);</code> Empty string becomes non-null object.
Unbox a known string	<code>s = (string)o;</code> Null object becomes null string. InvalidCastException if not a string.	<code>s = unbox_value<hstring>(o);</code> Null object crashes. Crash if not a string.
Unbox a possible string	<code>s = o as string;</code> Null object or non-string becomes null string. OR <code>s = o as string ?? fallback;</code> Null or non-string becomes fallback. Empty string preserved.	<code>s = unbox_value_or<hstring>(o, fallback);</code> Null or non-string becomes fallback. Empty string preserved.

Making a class available to the {Binding} markup extension

If you intend to use the {Binding} markup extension to data bind to your data type, then see [Binding object declared using {Binding}](#).

Consuming objects from XAML markup

In a C# project, you can consume private members and named elements from XAML markup. But in C++/WinRT, all entities consumed by using the XAML [{x:Bind} markup extension](#) must be exposed publicly in IDL.

Also, binding to a Boolean displays `true` or `false` in C#, but it shows

`Windows.Foundation.IReference`1<Boolean>` in C++/WinRT.

For more info, and code examples, see [Consuming objects from markup](#).

Making a data source available to XAML markup

In C++/WinRT version 2.0.190530.8 or later, [winrt::single_threaded_observable_vector](#) creates an observable vector that supports both [IObservableVector<T>](#) and [IObservableVector<IInspectable>](#). For an example, see [Porting the Scenarios property](#).

You can author your **Midl file (.idl)** like this (also see [Factoring runtime classes into Midl files \(.idl\)](#)).

idl

```
namespace Bookstore
{
    runtimeclass BookSku { ... }

    runtimeclass BookstoreViewModel
    {
        Windows.Foundation.Collections.IObservableVector<BookSku> BookSkus{ get; };
    }

    runtimeclass MainPage : Windows.UI.Xaml.Controls.Page
    {
        MainPage();
        BookstoreViewModel MainViewModel{ get; };
    }
}
```

```
}  
}
```

And implement like this.

C++/WinRT

```
// BookstoreViewModel.h  
...  
struct BookstoreViewModel : BookstoreViewModelT<BookstoreViewModel>  
{  
    BookstoreViewModel()  
    {  
        m_bookSkus = winrt::single_threaded_observable_vector<Bookstore::BookSku>();  
        m_bookSkus.Append(winrt::make<Bookstore::implementation::BookSku>(L"To Kill A  
Mockingbird"));  
    }  
  
    Windows::Foundation::Collections::IObservableVector<Bookstore::BookSku> BookSkus();  
    {  
        return m_bookSkus;  
    }  
  
private:  
    Windows::Foundation::Collections::IObservableVector<Bookstore::BookSku> m_bookSkus;  
};  
...
```

For more info, see [XAML items controls; bind to a C++/WinRT collection](#), and [Collections with C++/WinRT](#).

Making a data source available to XAML markup (prior to C++/WinRT 2.0.190530.8)

XAML data binding requires that an items source implements [IIterable<IInspectable>](#), as well as one of the following combinations of interfaces.

- [IObservableVector<IInspectable>](#)
- [IBindableVector](#) and [INotifyCollectionChanged](#)
- [IBindableVector](#) and [IBindableObservableVector](#)
- [IBindableVector](#) by itself (will not respond to changes)
- [IVector<IInspectable>](#)
- [IBindableIterable](#) (will iterate and save elements into a private collection)

A generic interface such as [IVector<T>](#) can't be detected at runtime. Each [IVector<T>](#) has a different interface identifier (IID), which is a function of T. Any developer can expand the set of T arbitrarily, so clearly the XAML binding code can never know the full set to query for. That restriction isn't a problem for C# because every CLR object that implements [IEnumerable<T>](#) automatically implements [IEnumerable](#). At the ABI level, that means that every object that implements [IObservableVector<T>](#) automatically implements [IObservableVector<IInspectable>](#).

C++/WinRT doesn't offer that guarantee. If a C++/WinRT runtime class implements [IObservableVector<T>](#), then we can't assume that an implementation of [IObservableVector<IInspectable>](#) is somehow also provided.

Consequently, here's how the previous example will need to look.

idl


```
...
runtimeclass BookstoreViewModel
{
    // This is really an observable vector of BookSku.
    Windows.Foundation.Collections.IObservableVector<Object> BookSkus{ get; };
}
```

And the implementation.

C++/WinRT

```
// BookstoreViewModel.h
...
struct BookstoreViewModel : BookstoreViewModelT<BookstoreViewModel>
{
    BookstoreViewModel()
    {
        m_bookSkus = winrt::single_threaded_observable_vector<Windows::Foundation::IIInspectable>
    };
    m_bookSkus.Append(winrt::make<Bookstore::implementation::BookSku>(L"To Kill A
Mockingbird"));
    }

    // This is really an observable vector of BookSku.
    Windows::Foundation::Collections::IObservableVector<Windows::Foundation::IIInspectable>
    BookSkus();
    {
        return m_bookSkus;
    }

private:
    Windows::Foundation::Collections::IObservableVector<Windows::Foundation::IIInspectable>
    m_bookSkus;
};
...
```

If you need to access objects in *m_bookSkus*, then you'll need to QI them back to **Bookstore::BookSku**.

C++/WinRT

```
Widget MyPage::BookstoreViewModel(winrt::hstring title)
{
    for (auto&& obj : m_bookSkus)
    {
        auto bookSku = obj.as<Bookstore::BookSku>();
        if (bookSku.Title() == title) return bookSku;
    }
    return nullptr;
}
```

Derived classes

In order to derive from a runtime class, the base class must be *composable*. C# doesn't require that you take any special steps to make your classes composable, but C++/WinRT does. You use the [unsealed keyword](#) to indicate that you want your class to be usable as a base class.

idl

```
unsealed runtimeclass BasePage : Windows.UI.Xaml.Controls.Page
{
```

```
    ...
}
runtimeclass DerivedPage : BasePage
{
    ...
}
```

In the header file for your [implementation type](#), you must include the base class header file before you include the autogenerated header for the derived class. Otherwise you'll get errors such as "Illegal use of this type as an expression".

C++/WinRT

```
// DerivedPage.h
#include "BasePage.h"           // This comes first.
#include "DerivedPage.g.h"      // Otherwise this header file will produce an error.

namespace winrt::MyNamespace::implementation
{
    struct DerivedPage : DerivedPageT<DerivedPage>
    {
        ...
    }
}
```

Important APIs

- [winrt namespace](#)

Related topics

- [C# tutorials](#)
- [C++/WinRT](#)
- [Data binding in depth](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Interop between C++/WinRT and the ABI

Article • 09/29/2023

This topic shows how to convert between SDK application binary interface (ABI) and C++/WinRT objects. You can use these techniques to interop between code that uses these two ways of programming with the Windows Runtime, or you can use them as you gradually move your code from the ABI to C++/WinRT.

In general, C++/WinRT exposes ABI types as `void*`, so that you don't need to include platform header files.

ⓘ Note

In the code examples, we use `reinterpret_cast` (rather than `static_cast`) in an effort to *telegraph* what are inherently unsafe casts.

What is the Windows Runtime ABI, and what are ABI types?

A Windows Runtime class (runtime class) is really an abstraction. This abstraction defines a binary interface (the Application Binary Interface, or ABI) that allows various programming languages to interact with an object. Regardless of programming language, client code interaction with a Windows Runtime object happens at the lowest level, with client language constructs translated into calls into the object's ABI.

The Windows SDK headers in the folder "%WindowsSdkDir%Include\10.0.17134.0\winrt" (adjust the SDK version number for your case, if necessary), are the Windows Runtime ABI header files. They were produced by the MIDL compiler. Here's an example of including one of these headers.

C++

```
#include <windows.foundation.h>
```

And here's a simplified example of one of the ABI types that you'll find in that particular SDK header. Note the **ABI** namespace; **Windows::Foundation**, and all other Windows namespaces, are declared by the SDK headers within the **ABI** namespace.

C++

```
namespace ABI::Windows::Foundation
{
    IUriRuntimeClass : public IInspectable
    {
    public:
        /* [propget] */ virtual HRESULT STDMETHODCALLTYPE get_AbsoluteUri(/*
[retval, out] */ __RPC__deref_out_opt HSTRING * value) = 0;
        ...
    }
}
```

IUriRuntimeClass is a COM interface. But more than that—since its base is **IInspectable**—**IUriRuntimeClass** is a Windows Runtime interface. Note the **HRESULT** return type, rather than the raising of exceptions. And the use of artifacts such as the **HSTRING** handle (it's good practice to set that handle back to `nullptr` when you're finished with it). This gives a taste of what the Windows Runtime looks like at the application binary level; in other words, at the COM programming level.

The Windows Runtime is based on Component Object Model (COM) APIs. You can access the Windows Runtime that way, or you can access it through *language projections*. A projection hides the COM details, and provides a more natural programming experience for a given language.

For example, if you look in the folder "%WindowsSdkDir%Include\10.0.17134.0\cppwinrt\winrt" (again, adjust the SDK version number for your case, if necessary), then you'll find the C++/WinRT language projection headers. There's a header for each Windows namespace, just like there's one ABI header per Windows namespace. Here's an example of including one of the C++/WinRT headers.

C++/WinRT

```
#include <winrt/Windows.Foundation.h>
```

And, from that header, here (simplified) is the C++/WinRT equivalent of that ABI type we just saw.

C++/WinRT

```
namespace winrt::Windows::Foundation
{
    struct Uri : IUriRuntimeClass, ...
    {
        winrt::hstring AbsoluteUri() const { ... }
    }
}
```

```
    ...  
};  
}
```

The interface here is modern, standard C++. It does away with **HRESULTS** (C++/WinRT raises exceptions if necessary). And the accessor function returns a simple string object, which is cleaned up at the end of its scope.

This topic is for cases when you want to interop with, or port, code that works at the Application Binary Interface (ABI) layer.

Converting to and from ABI types in code

For safety and simplicity, for conversions in both directions you can simply use [winrt::com_ptr](#), [com_ptr::as](#), and [winrt::Windows::Foundation::IUnknown::as](#). Here's a code example (based on the **Console App** project template), which also illustrates how you can use namespace aliases for the different islands to deal with otherwise potential namespace collisions between the C++/WinRT projection and the ABI.

C++/WinRT

```
// pch.h  
#pragma once  
#include <windows.foundation.h>  
#include <unknwn.h>  
#include "winrt/Windows.Foundation.h"  
  
// main.cpp  
#include "pch.h"  
  
namespace winrt  
{  
    using namespace Windows::Foundation;  
}  
  
namespace abi  
{  
    using namespace ABI::Windows::Foundation;  
};  
  
int main()  
{  
    winrt::init_apartment();  
  
    winrt::Uri uri(L"http://aka.ms/cppwinrt");  
  
    // Convert to an ABI type.  
    winrt::com_ptr<abi::IStringable> ptr{ uri.as<abi::IStringable>() };  
}
```

```

// Convert from an ABI type.
uri = ptr.as<winrt::Uri>();
winrt::IStringable uriAsIStringable{ ptr.as<winrt::IStringable>() };
}

```

The implementations of the `as` functions call [QueryInterface](#). If you want lower-level conversions that only call [AddRef](#), then you can use the [winrt::copy_to_abi](#) and [winrt::copy_from_abi](#) helper functions. This next code example adds these lower-level conversions to the code example above.

Important

When interoperating with ABI types it's critical that the ABI type used corresponds to the default interface of the C++/WinRT object. Otherwise, invocations of methods on the ABI type will actually end up calling methods in the same vtable slot on the default interface with very unexpected results. Note that [winrt::copy_to_abi](#) does not protect against this at compile time since it uses `void*` for all ABI types and assumes that the caller has been careful not to mis-match the types. This is to avoid requiring C++/WinRT headers to reference ABI headers when ABI types may never be used.

C++/WinRT

```

int main()
{
    // The code in main() already shown above remains here.

    // Lower-level conversions that only call AddRef.

    // Convert to an ABI type.
    ptr = nullptr;
    winrt::copy_to_abi(uriAsIStringable, *ptr.put_void());

    // Convert from an ABI type.
    uri = nullptr;
    winrt::copy_from_abi(uriAsIStringable, ptr.get());
    ptr = nullptr;
}

```

Here are other similarly low-level conversions techniques but using raw pointers to ABI interface types (those defined by the Windows SDK headers) this time.

C++/WinRT

```

// The code in main() already shown above remains here.

```

```

// Copy to an owning raw ABI pointer with copy_to_abi.
abi::IStringable* owning{ nullptr };
winrt::copy_to_abi(uriAsIStringable, *reinterpret_cast<void**>
(&owning));

// Copy from a raw ABI pointer.
uri = nullptr;
winrt::copy_from_abi(uriAsIStringable, owning);
owning->Release();

```

For the lowest-level conversions, which only copy addresses, you can use the [winrt::get_abi](#), [winrt::detach_abi](#), and [winrt::attach_abi](#) helper functions.

`WINRT_ASSERT` is a macro definition, and it expands to `_ASSERTE`.

C++/WinRT

```

// The code in main() already shown above remains here.

// Lowest-level conversions that only copy addresses

// Convert to a non-owning ABI object with get_abi.
abi::IStringable* non_owning{ reinterpret_cast<abi::IStringable*>
(winrt::get_abi(uriAsIStringable)) };
WINRT_ASSERT(non_owning);

// Avoid interlocks this way.
owning = reinterpret_cast<abi::IStringable*>
(winrt::detach_abi(uriAsIStringable));
WINRT_ASSERT(!uriAsIStringable);
winrt::attach_abi(uriAsIStringable, owning);
WINRT_ASSERT(uriAsIStringable);

```

convert_from_abi function

This helper function converts a raw ABI interface pointer to an equivalent C++/WinRT object, with minimal overhead.

C++/WinRT

```

template <typename T>
T convert_from_abi(::IUnknown* from)
{
    T to{ nullptr }; // `T` is a projected type.

    winrt::check_hresult(from->QueryInterface(winrt::guid_of<T>(),
        winrt::put_abi(to)));
}

```

```
    return to;
}
```

The function simply calls [QueryInterface](#) to query for the default interface of the requested C++/WinRT type.

As we've seen, a helper function is not required to convert from a C++/WinRT object to the equivalent ABI interface pointer. Simply use the [winrt::Windows::Foundation::IUnknown::as](#) (or [try_as](#)) member function to query for the requested interface. The `as` and `try_as` functions return a [winrt::com_ptr](#) object wrapping the requested ABI type.

Code example using `convert_from_abi`

Here's a code example showing this helper function in practice.

C++/WinRT

```
// pch.h
#pragma once
#include <windows.foundation.h>
#include <unknwn.h>
#include "winrt/Windows.Foundation.h"

// main.cpp
#include "pch.h"
#include <iostream>

using namespace winrt;
using namespace Windows::Foundation;

namespace winrt
{
    using namespace Windows::Foundation;
}

namespace abi
{
    using namespace ABI::Windows::Foundation;
};

namespace sample
{
    template <typename T>
    T convert_from_abi(::IUnknown* from)
    {
        T to{ nullptr }; // `T` is a projected type.

        winrt::check_hresult(from->QueryInterface(winrt::guid_of<T>()),
```



```

        winrt::put_abi(to));

        return to;
    }
    inline auto put_abi(winrt::hstring& object) noexcept
    {
        return reinterpret_cast<HSTRING*>(winrt::put_abi(object));
    }
}

int main()
{
    winrt::init_apartment();

    winrt::Uri uri(L"http://aka.ms/cppwinrt");
    std::wcout << "C++/WinRT: " << uri.Domain().c_str() << std::endl;

    // Convert to an ABI type.
    winrt::com_ptr<abi::IUriRuntimeClass> ptr =
uri.as<abi::IUriRuntimeClass>();
    winrt::hstring domain;
    winrt::check_hresult(ptr->get_Domain(sample::put_abi(domain)));
    std::wcout << "ABI: " << domain.c_str() << std::endl;

    // Convert from an ABI type.
    winrt::Uri uri_from_abi = sample::convert_from_abi<winrt::Uri>
(ptr.get());

    WINRT_ASSERT(uri.Domain() == uri_from_abi.Domain());
    WINRT_ASSERT(uri == uri_from_abi);
}

```

Interoperating with ABI COM interface pointers

The helper function template below illustrates how to copy an ABI COM interface pointer of a given type to its equivalent C++/WinRT projected smart pointer type.

C++/WinRT

```

template<typename To, typename From>
To to_winrt(From* ptr)
{
    To result{ nullptr };
    winrt::check_hresult(ptr->QueryInterface(winrt::guid_of<To>(),
winrt::put_abi(result)));
    return result;
}

...
ID2D1Factory1* com_ptr{ ... };
auto cppwinrt_ptr {to_winrt<winrt::com_ptr<ID2D1Factory1>>(com_ptr)};

```

This next helper function template is equivalent, except that it copies from the smart pointer type from the [Windows Implementation Libraries \(WIL\)](#).

C++/WinRT

```
template<typename To, typename From, typename ErrorPolicy>
To to_winrt(wil::com_ptr_t<From, ErrorPolicy> const& ptr)
{
    To result{ nullptr };
    if constexpr (std::is_same_v<typename ErrorPolicy::result, void>)
    {
        ptr.query_to(winrt::guid_of<To>(), winrt::put_abi(result));
    }
    else
    {
        winrt::check_result(ptr.query_to(winrt::guid_of<To>(),
winrt::put_abi(result)));
    }
    return result;
}
```

Also see [Consume COM components with C++/WinRT](#).

Unsafe interop with ABI COM interface pointers

The table that follows shows (in addition to other operations) unsafe conversions between an ABI COM interface pointer of a given type and its equivalent C++/WinRT projected smart pointer type. For the code in the table, assume these declarations.

C++/WinRT

```
winrt::Sample s;
ISample* p;

void GetSample(_Out_ ISample** pp);
```

Assume further that **ISample** is the default interface for **Sample**.

You can assert that at compile time with this code.

C++/WinRT

```
static_assert(std::is_same_v<winrt::default_interface<winrt::Sample>,
winrt::ISample>);
```

Operation	How to do it	Notes
Extract ISample* from winrt::Sample	<code>p = reinterpret_cast<ISample*>(get_abi(s));</code>	<i>s</i> still owns the object.
Detach ISample* from winrt::Sample	<code>p = reinterpret_cast<ISample*>(detach_abi(s));</code>	<i>s</i> no longer owns the object.
Transfer ISample* to new winrt::Sample	<code>winrt::Sample s{ p, winrt::take_ownership_from_abi };</code>	<i>s</i> takes ownership of the object.
Set ISample* into winrt::Sample	<code>*put_abi(s) = p;</code>	<i>s</i> takes ownership of the object. Any object previously owned by <i>s</i> is leaked (will assert in debug).
Receive ISample* into winrt::Sample	<code>GetSample(reinterpret_cast<ISample**>(put_abi(s)));</code>	<i>s</i> takes ownership of the object. Any object previously owned by <i>s</i> is leaked (will assert in debug).
Replace ISample* in winrt::Sample	<code>attach_abi(s, p);</code>	<i>s</i> takes ownership of the object. The object previously owned by <i>s</i> is freed.
Copy ISample* to winrt::Sample	<code>copy_from_abi(s, p);</code>	<i>s</i> makes a new reference to the object. The object previously owned by <i>s</i> is freed.
Copy winrt::Sample to ISample*	<code>copy_to_abi(s, reinterpret_cast<void*&>(p));</code>	<i>p</i> receives a copy of the object. Any object previously owned by <i>p</i> is leaked.

Interoperating with the ABI's GUID struct

GUID (`/previous-versions/aa373931(v%3Dvs.80)`) is projected as **winrt::guid**. For APIs that you implement, you must use **winrt::guid** for GUID parameters. Otherwise, there are automatic conversions between **winrt::guid** and **GUID** as long as you include `unknwn.h` (implicitly included by `<windows.h>` and many other header files) before you include any C++/WinRT headers.

If you don't do that, then you can hard-`reinterpret_cast` between them. For the table that follows, assume these declarations.

```
winrt::guid winrtguid;
GUID abiguid;
```

Conversion	With <code>#include <unkwn.h></code>	Without <code>#include <unkwn.h></code>
From <code>winrt::guid</code> to <code>GUID</code>	<code>abiguid = winrtguid;</code>	<code>abiguid = reinterpret_cast<GUID*>(winrtguid);</code>
From <code>GUID</code> to <code>winrt::guid</code>	<code>winrtguid = abiguid;</code>	<code>winrtguid = reinterpret_cast<winrt::guid*>(abiguid);</code>

You can construct a `winrt::guid` like this.

C++/WinRT

```
winrt::guid myGuid{ 0xC380465D, 0x2271, 0x428C, { 0x9B, 0x83, 0xEC, 0xEA,
0x3B, 0x4A, 0x85, 0xC1} };
```

For a gist showing how to construct a `winrt::guid` from a string, see [make_guid.cpp](#).

Interoperating with the ABI's HSTRING

The table that follows shows conversions between `winrt::hstring` and `HSTRING`, and other operations. For the code in the table, assume these declarations.

C++/WinRT

```
winrt::hstring s;
HSTRING h;

void GetString(_Out_ HSTRING* value);
```

Operation	How to do it	Notes
Extract <code>HSTRING</code> from <code>hstring</code>	<code>h = reinterpret_cast<HSTRING*>(get_abi(s));</code>	<code>s</code> still owns the string.
Detach <code>HSTRING</code> from <code>hstring</code>	<code>h = reinterpret_cast<HSTRING*>(detach_abi(s));</code>	<code>s</code> no longer owns the string.
Set <code>HSTRING</code> into <code>hstring</code>	<code>*put_abi(s) = h;</code>	<code>s</code> takes ownership of string. Any string previously owned by <code>s</code> is

Operation	How to do it	Notes
		leaked (will assert in debug).
Receive HSTRING into hstring	<code>GetString(reinterpret_cast<HSTRING*>(put_abi(s)));</code>	<i>s</i> takes ownership of string. Any string previously owned by <i>s</i> is leaked (will assert in debug).
Replace HSTRING in hstring	<code>attach_abi(s, h);</code>	<i>s</i> takes ownership of string. The string previously owned by <i>s</i> is freed.
Copy HSTRING to hstring	<code>copy_from_abi(s, h);</code>	<i>s</i> makes a private copy of the string. The string previously owned by <i>s</i> is freed.
Copy hstring to HSTRING	<code>copy_to_abi(s, reinterpret_cast<void*>(h));</code>	<i>h</i> receives a copy of the string. Any string previously owned by <i>h</i> is leaked.

In addition, the Windows Implementation Libraries (WIL) [string helpers](#) perform basic string manipulations. To use the WIL string helpers, include [<wil/resource.h>](#), and refer to the table below. Follow the links in the table for full details.

Operation	WIL string helper for more info
Provide a raw Unicode or ANSI string pointer and an optional length; obtain a suitably-specialized unique_any wrapper	wil::make_something_string
Unwrap a smart object until a raw null-terminated Unicode string pointer is found	wil::str_raw_ptr
Obtain the string wrapped by a smart pointer object; or the empty string <code>L""</code> if the smart pointer is empty	wil::string_get_not_null
Concatenate any number of strings	wil::str_concat
Obtain a string from a printf-style format string and a corresponding parameter list	wil::str_printf

Important APIs

- [AddRef](#) function
- [QueryInterface](#) function
- [winrt::attach_abi](#) function
- [winrt::com_ptr](#) struct template
- [winrt::copy_from_abi](#) function

- `winrt::copy_to_abi` function
- `winrt::detach_abi` function
- `winrt::get_abi` function
- `winrt::Windows::Foundation::IUnknown::as` member function
- `winrt::Windows::Foundation::IUnknown::try_as` member function

Strong and weak references in C++/WinRT

Article • 10/20/2022

The Windows Runtime is a reference-counted system; and in such a system it's important for you to know about the significance of, and distinction between, strong and weak references (and references that are neither, such as the implicit *this* pointer). As you'll see in this topic, knowing how to manage these references correctly can mean the difference between a reliable system that runs smoothly, and one that crashes unpredictably. By providing helper functions that have deep support in the language projection, [C++/WinRT](#) meets you halfway in your work of building more complex systems simply and correctly.

ⓘ Note

With only a few exceptions, weak reference support is on by default for Windows Runtime types that you consume or author in [C++/WinRT](#).

Windows.UI.Composition and **Windows.Devices.Input.PenDevice** are examples of exceptions—that is, namespaces where weak reference support is *not* on for those types. Also see [If your auto-revoke delegate fails to register](#).

If you're authoring types, then see the [Weak references in C++/WinRT](#) section in this topic.

Safely accessing the *this* pointer in a class-member coroutine

For more info about coroutines, and code examples, see [Concurrency and asynchronous operations with C++/WinRT](#).

The code listing below shows a typical example of a coroutine that's a member function of a class. You can copy-paste this example into the specified files in a new **Windows Console Application (C++/WinRT)** project.

C++/WinRT

```
// pch.h
#pragma once
#include <iostream>
#include <winrt/Windows.Foundation.h>
```

```

// main.cpp : Defines the entry point for the console application.
#include "pch.h"

using namespace winrt;
using namespace Windows::Foundation;
using namespace std::chrono_literals;

struct MyClass : winrt::implements<MyClass, IInspectable>
{
    winrt::hstring m_value{ L"Hello, World!" };

    IAsyncOperation<winrt::hstring> RetrieveValueAsync()
    {
        co_await 5s;
        co_return m_value;
    }
};

int main()
{
    winrt::init_apartment();

    auto myclass_instance{ winrt::make_self<MyClass>() };
    auto async{ myclass_instance->RetrieveValueAsync() };

    winrt::hstring result{ async.get() };
    std::wcout << result.c_str() << std::endl;
}

```

MyClass::RetrieveValueAsync spends some time working, and eventually it returns a copy of the `MyClass::m_value` data member. Calling **RetrieveValueAsync** causes an asynchronous object to be created, and that object has an implicit *this* pointer (through which, eventually, `m_value` is accessed).

Remember that, in a coroutine, execution is synchronous up until the first suspension point, where control is returned to the caller. In **RetrieveValueAsync**, the first `co_await` is the first suspension point. By the time the coroutine resumes (around five seconds later, in this case), anything might have happened to the implicit *this* pointer through which we access `m_value`.

Here's the full sequence of events.

1. In **main**, an instance of **MyClass** is created (`myclass_instance`).
2. The `async` object is created, pointing (via its *this*) to `myclass_instance`.
3. The **winrt::Windows::Foundation::IAsyncAction::get** function hits its first suspension point, blocks for a few seconds, and then returns the result of **RetrieveValueAsync**.

4. **RetrieveValueAsync** returns the value of `this->m_value`.

Step 4 is safe only as long as *this* remains valid.

But what if the class instance is destroyed before the async operation completes? There are all kinds of ways the class instance could go out of scope before the asynchronous method has completed. But we can simulate it by setting the class instance to `nullptr`.

C++/WinRT

```
int main()
{
    winrt::init_apartment();

    auto myclass_instance{ winrt::make_self<MyClass>() };
    auto async{ myclass_instance->RetrieveValueAsync() };
    myclass_instance = nullptr; // Simulate the class instance going out of
    scope.

    winrt::hstring result{ async.get() }; // Behavior is now undefined;
    crashing is likely.
    std::wcout << result.c_str() << std::endl;
}
```

After the point where we destroy the class instance, it looks like we don't directly refer to it again. But of course the asynchronous object has a *this* pointer to it, and tries to use that to copy the value stored inside the class instance. The coroutine is a member function, and it expects to be able to use its *this* pointer with impunity.

With this change to the code, we run into a problem in step 4, because the class instance has been destroyed, and *this* is no longer valid. As soon as the asynchronous object attempts to access the variable inside the class instance, it will crash (or do something entirely undefined).

The solution is to give the asynchronous operation—the coroutine—its own strong reference to the class instance. As currently written, the coroutine effectively holds a raw *this* pointer to the class instance; but that's not enough to keep the class instance alive.

To keep the class instance alive, change the implementation of **RetrieveValueAsync** to that shown below.

C++/WinRT

```
IAsyncOperation<winrt::hstring> RetrieveValueAsync()
{
    auto strong_this{ get_strong() }; // Keep *this* alive.
    co_await 5s;
}
```

```
co_return m_value;
}
```

A C++/WinRT class directly or indirectly derives from the [winrt::implements](#) template. Because of that, the C++/WinRT object can call its [implements::get_strong](#) protected member function to retrieve a strong reference to its *this* pointer. Note that there's no need to actually use the `strong_this` variable in the code example above; simply calling **get_strong** increments the C++/WinRT object's reference count, and keeps its implicit *this* pointer valid.

Important

Because **get_strong** is a member function of the **winrt::implements** struct template, you can call it only from a class that directly or indirectly derives from **winrt::implements**, such as a C++/WinRT class. For more info about deriving from **winrt::implements**, and examples, see [Author APIs with C++/WinRT](#).

This resolves the problem that we previously had when we got to step 4. Even if all other references to the class instance disappear, the coroutine has taken the precaution of guaranteeing that its dependencies are stable.

If a strong reference isn't appropriate, then you can instead call [implements::get_weak](#) to retrieve a weak reference to *this*. Just confirm that you can retrieve a strong reference before accessing *this*. Again, **get_weak** is a member function of the **winrt::implements** struct template.

C++/WinRT

```
IAsyncOperation<winrt::hstring> RetrieveValueAsync()
{
    auto weak_this{ get_weak() }; // Maybe keep *this* alive.

    co_await 5s;

    if (auto strong_this{ weak_this.get() })
    {
        co_return m_value;
    }
    else
    {
        co_return L"";
    }
}
```

In the example above, the weak reference doesn't keep the class instance from being destroyed when no strong references remain. But it gives you a way of checking whether a strong reference can be acquired before accessing the member variable.

Safely accessing the *this* pointer with an event-handling delegate

The scenario

For general info about event-handling, see [Handle events by using delegates in C++/WinRT](#).

The previous section highlighted potential lifetime issues in the areas of coroutines and concurrency. But, if you handle an event with an object's member function, or from within a lambda function inside an object's member function, then you need to think about the relative lifetimes of the event recipient (the object handling the event) and the event source (the object raising the event). Let's look at some code examples.

The code listing below first defines a simple **EventSource** class, which raises a generic event that's handled by any delegates that have been added to it. This example event happens to use the [Windows::Foundation::EventHandler](#) delegate type, but the issues and remedies here apply to any and all delegate types.

Then, the **EventRecipient** class provides a handler for the **EventSource::Event** event in the form of a lambda function.

C++/WinRT

```
// pch.h
#pragma once
#include <iostream>
#include <winrt/Windows.Foundation.h>

// main.cpp : Defines the entry point for the console application.
#include "pch.h"

using namespace winrt;
using namespace Windows::Foundation;

struct EventSource
{
    winrt::event<EventHandler<int>> m_event;

    void Event(EventHandler<int> const& handler)
    {
```

```

        m_event.add(handler);
    }

    void RaiseEvent()
    {
        m_event(nullptr, 0);
    }
};

struct EventRecipient : winrt::implements<EventRecipient, IInspectable>
{
    winrt::hstring m_value{ L"Hello, World!" };

    void Register(EventSource& event_source)
    {
        event_source.Event([&](auto&& ...)
        {
            std::wcout << m_value.c_str() << std::endl;
        }));
    }
};

int main()
{
    winrt::init_apartment();

    EventSource event_source;
    auto event_recipient{ winrt::make_self<EventRecipient>() };
    event_recipient->Register(event_source);
    event_source.RaiseEvent();
}

```

The pattern is that the event recipient has a lambda event handler with dependencies on its *this* pointer. Whenever the event recipient outlives the event source, it outlives those dependencies. And in those cases, which are common, the pattern works well. Some of these cases are obvious, such as when a UI page handles an event raised by a control that's on the page. The page outlives the button—so, the handler also outlives the button. This holds true any time the recipient owns the source (as a data member, for example), or any time the recipient and the source are siblings and directly owned by some other object.

When you're sure you have a case where the handler won't outlive the *this* that it depends on, then you can capture *this* normally, without consideration for strong or weak lifetime.

But there are still cases where *this* doesn't outlive its use in a handler (including handlers for completion and progress events raised by asynchronous actions and operations), and it's important to know how to deal with them.

- When an event source raises its events *synchronously*, you can revoke your handler and be confident that you won't receive any more events. But for asynchronous events, even after revoking (and especially when revoking within the destructor), an in-flight event might reach your object after it has started destructing. Finding a place to unsubscribe prior to destruction might mitigate the issue, but continue reading for a robust solution.
- If you're authoring a coroutine to implement an asynchronous method, then it's possible.
- In rare cases with certain XAML UI framework objects ([SwapChainPanel](#), for example), then it's possible, if the recipient is finalized without unregistering from the event source.

The issue

This next version of the **main** function simulates what happens when the event recipient is destroyed (perhaps it goes out of scope) while the event source is still raising events.

C++/WinRT

```
int main()
{
    winrt::init_apartment();

    EventSource event_source;
    auto event_recipient{ winrt::make_self<EventRecipient>() };
    event_recipient->Register(event_source);
    event_recipient = nullptr; // Simulate the event recipient going out of
    scope.
    event_source.RaiseEvent(); // Behavior is now undefined within the
    lambda event handler; crashing is likely.
}
```

The event recipient is destroyed, but the lambda event handler within it is still subscribed to the **Event** event. When that event is raised, the lambda attempts to dereference the *this* pointer, which is at that point invalid. So, an access violation results from code in the handler (or in a coroutine's continuation) attempting to use it.

❗ Important

If you encounter a situation like this, then you'll need to think about the lifetime of the *this* object; and whether or not the captured *this* object outlives the capture. If it doesn't, then capture it with a strong or a weak reference, as we'll demonstrate below.